



Universidad Tecnológica de Bolívar

Universidad Tecnológica de Bolívar

Estimación del coeficiente de restitución

Actividad 1 — Segundo corte

Sensado y Modelado de Sistemas Físicos

Estudiantes:

German De Armas Castaño

Angel Vega Rodriguez

Michael Taboada Naranjo

Profesor:

David Sierra Porta

23 de abril de 2026

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Objetivos	2
2. Fundamentos Teóricos	3
2.1. El coeficiente de restitución (e)	3
2.2. Espacio de color HSV y segmentación	3
2.3. Operaciones morfológicas	4
2.4. Cálculo del centroide mediante momentos espaciales	4
2.5. Detección de picos con <code>scipy.signal.find_peaks</code>	4
3. Implementación en el Notebook	5
3.1. Módulo 1: Detección del objeto (<code>detect_ball</code>)	5
3.2. Módulo 2: Calibración interactiva HSV	6
3.3. Módulo 3: Tracking frame-a-frame	6
3.4. Módulo 4: Cálculo del coeficiente de restitución	7
3.4.1. Paso 1: Determinación del nivel del suelo	7
3.4.2. Paso 2: Conversión a alturas relativas	7
3.4.3. Paso 3: Detección de picos	7
3.4.4. Paso 4: Cálculo iterativo de e	7
3.5. Módulo 5: Visualización de resultados	8
4. Aplicación Web en el Bento Launcher	8
4.1. Arquitectura	9
4.2. Lanzamiento desde el Bento Launcher	9
4.3. Carga del video	10
4.4. Calibración HSV interactiva	10
4.5. Procesamiento y tracking	11
4.6. Resultados	11
5. Discusión	12
5.1. Ventajas del enfoque computacional	12
5.2. Limitaciones y fuentes de error	12
6. Conclusiones	13

1. Introducción

El presente reporte documenta el diseño, implementación y validación de un sistema automatizado para la estimación experimental del **coeficiente de restitución** (e) de un cuerpo que colisiona repetidamente contra una superficie rígida. El proyecto se desarrolló en dos fases claramente diferenciadas:

1. **Fase de exploración y prototipado** — implementada íntegramente en un *notebook* de Python (`restitution_calculator.ipynb`), donde se diseñaron, probaron y validaron todos los algoritmos de visión computacional y procesamiento de señales.
2. **Fase de productización** — los hallazgos del notebook se encapsularon en una aplicación web interactiva (React + FastAPI) integrada al ecosistema **Bento Launcher**, permitiendo que cualquier usuario reproduzca el análisis sin necesidad de escribir código.

1.1. Planteamiento del problema

El coeficiente de restitución cuantifica la fracción de energía cinética que se conserva tras un impacto. Medirlo manualmente — observando a simple vista las alturas sucesivas de un objeto que rebota — introduce errores significativos debido a:

- La **latencia de reacción humana**: el instante de altura máxima dura fracciones de segundo.
- El **sesgo de perspectiva**: la posición del observador distorsiona la lectura de alturas.
- La **falta de repetibilidad**: cada observador obtiene valores distintos.

Un sistema de visión computacional elimina estas fuentes de error al evaluar *cada fotograma* del video, extrayendo la posición vertical del objeto con resolución sub-píxel y consistencia absoluta entre ejecuciones.

1.2. Objetivos

1. Implementar un algoritmo de segmentación por color en el espacio HSV para aislar el objeto en cada fotograma del video.
2. Extraer la serie temporal de la coordenada vertical (Y) del centroide del objeto a lo largo de todo el video.
3. Aplicar detección de picos (`scipy.signal.find_peaks`) con filtrado por prominencia para identificar las alturas máximas de cada rebote.
4. Calcular el coeficiente de restitución promedio $\bar{e}$ y su desviación estándar σ_e a partir de las alturas consecutivas.
5. Encapsular el pipeline completo en una aplicación web desplegable desde el Bento Launcher.

2. Fundamentos Teóricos

2.1. El coeficiente de restitución (e)

El coeficiente de restitución es un escalar adimensional que relaciona las velocidades relativas antes y después de una colisión. En su forma general (Newton):

$$e = \frac{v_{\text{sep}}}{v_{\text{aprox}}} \quad (1)$$

donde v_{sep} es la velocidad relativa de separación y v_{aprox} la de aproximación. Para un objeto que rebota verticalmente contra el suelo, la conservación de energía mecánica (con disipación únicamente en el impacto) permite expresar e en función de las alturas:

$$e = \sqrt{\frac{h_{n+1}}{h_n}} \quad (2)$$

donde h_n es la altura máxima alcanzada en el rebote n -ésimo y h_{n+1} la del rebote siguiente. Esta relación se deriva directamente de la equivalencia entre energía potencial gravitatoria y energía cinética en el punto de impacto:

$$v_n = \sqrt{2gh_n} \quad (3)$$

$$v_{n+1} = \sqrt{2gh_{n+1}} \quad (4)$$

$$e = \frac{v_{n+1}}{v_n} = \sqrt{\frac{h_{n+1}}{h_n}} \quad (5)$$

El coeficiente e toma valores en el intervalo $[0, 1]$:

- $e = 0$: colisión perfectamente inelástica (toda la energía cinética se disipa).
- $e = 1$: colisión perfectamente elástica (no hay pérdida de energía).

Para un conjunto de N rebotes detectados, se calcula el promedio:

$$\bar{e} = \frac{1}{N-1} \sum_{i=1}^{N-1} \sqrt{\frac{h_{i+1}}{h_i}} \quad (6)$$

2.2. Espacio de color HSV y segmentación

El espacio de color HSV (Hue, Saturation, Value) descompone el color en tres componentes perceptualmente intuitivos:

- **Hue (H)**: el tono cromático, representado como un ángulo en $[0, 360]$ (en OpenCV se escala a $[0, 180]$).

- **Saturation (S)**: la pureza del color, en $[0, 255]$.
- **Value (V)**: la luminosidad, en $[0, 255]$.

A diferencia del espacio BGR/RGB, HSV permite aislar objetos por su tonalidad independientemente de las condiciones de iluminación, lo que lo hace ideal para la segmentación por color en escenas con iluminación variable.

La segmentación se realiza mediante un umbralizado por rango:

$$\text{mask}(x, y) = \begin{cases} 255 & \text{si } H_{\min} \leq H(x, y) \leq H_{\max} \wedge S_{\min} \leq S(x, y) \leq S_{\max} \wedge V_{\min} \leq V(x, y) \leq V_{\max} \\ 0 & \text{en caso contrario} \end{cases} \quad (7)$$

2.3. Operaciones morfológicas

Tras la segmentación, la máscara binaria suele contener ruido (píxeles aislados falsos positivos) y huecos internos. Se aplican dos operaciones morfológicas secuenciales:

1. **Apertura (MORPH_OPEN)**: erosión seguida de dilatación. Elimina pequeños objetos espurios sin alterar significativamente el contorno del objeto principal.
2. **Cierre (MORPH_CLOSE)**: dilatación seguida de erosión. Rellena huecos internos en la máscara del objeto.

Ambas operaciones utilizan un elemento estructurante elíptico de 5×5 píxeles.

2.4. Cálculo del centroide mediante momentos espaciales

Una vez obtenida la máscara limpia, se calculan los **momentos espaciales** de la región detectada. El centroide (c_x, c_y) se obtiene como:

$$c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}} \quad (8)$$

donde $M_{pq} = \sum_x \sum_y x^p \cdot y^q \cdot I(x, y)$ son los momentos de orden (p, q) de la imagen binaria I .

2.5. Detección de picos con `scipy.signal.find_peaks`

La serie temporal de alturas $h(t)$ se analiza con el algoritmo `find_peaks` de SciPy, que identifica máximos locales sujetos a restricciones configurables:

- **Prominencia (prominence)**: altura mínima que un pico debe sobresalir respecto a sus valles adyacentes. Filtra oscilaciones menores causadas por ruido en la detección.
- **Distancia (distance)**: separación mínima (en muestras) entre picos consecutivos. Evita la detección de picos duplicados en un mismo rebote.

3. Implementación en el Notebook

El notebook `restitution_calculator.ipynb` estructura el análisis en cinco módulos funcionales que se ejecutan secuencialmente. A continuación se detalla cada uno.

3.1. Módulo 1: Detección del objeto (`detect_ball`)

La función `detect_ball` recibe un fotograma y los umbrales HSV, y retorna el centroide del objeto detectado. El pipeline interno es:

1. Conversión del fotograma de BGR a HSV mediante `cv2.cvtColor`.
2. Generación de la máscara binaria con `cv2.inRange`.
3. Limpieza morfológica: apertura y cierre con kernel elíptico 5×5 .
4. Búsqueda de contornos con `cv2.findContours`.
5. Selección del contorno de mayor área (descartando áreas menores a `min_area= 50` px).
6. Cálculo del centroide (c_x, c_y) mediante momentos espaciales.

```

1 def detect_ball(frame, lower_hsv, upper_hsv, min_area=50):
2     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
3     mask = cv2.inRange(hsv, lower_hsv, upper_hsv)
4
5     kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
6     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
7     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
8
9     contours, _ = cv2.findContours(
10         mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
11     if not contours:
12         return None, mask
13
14     largest = max(contours, key=cv2.contourArea)
15     if cv2.contourArea(largest) < min_area:
16         return None, mask
17
18     M = cv2.moments(largest)
19     if M["m00"] == 0:
20         return None, mask
21
22     cx = int(M["m10"] / M["m00"])
23     cy = int(M["m01"] / M["m00"])
24     return (cx, cy), mask

```

Listing 1: Función de detección del objeto.

3.2. Módulo 2: Calibración interactiva HSV

Antes de procesar el video completo, es necesario determinar los umbrales HSV que aíslan correctamente el objeto. El notebook implementa una interfaz interactiva con `cv2.createTrackbar` que permite:

- Navegar por los fotogramas del video con un slider temporal.
- Ajustar en tiempo real los seis parámetros ($H_{\min}$, $H_{\max}$, $S_{\min}$, $S_{\max}$, $V_{\min}$, $V_{\max}$).
- Visualizar simultáneamente el fotograma original y la máscara binaria resultante.
- Confirmar la selección con la tecla ENTER.

Este paso es crítico: una calibración deficiente produce falsos positivos (ruido) o falsos negativos (pérdida del objeto), ambos degradando la calidad de la serie temporal.

3.3. Módulo 3: Tracking frame-a-frame

La función `process_video_tracking` recorre el video completo aplicando `detect_ball` en cada fotograma. Para cada detección exitosa, registra:

- El instante temporal $t = \text{frame_idx} / \text{fps}$.
- La coordenada vertical Y del centroide.

El resultado es un par de arrays NumPy: `time_series` y `y_series`, que conforman la señal cruda de posición vertical vs. tiempo.

```

1 def process_video_tracking(video_path, lower_hsv, upper_hsv):
2     cap = cv2.VideoCapture(video_path)
3     fps = cap.get(cv2.CAP_PROP_FPS)
4     time_series, y_series = [], []
5
6     frame_idx = 0
7     while True:
8         ret, frame = cap.read()
9         if not ret:
10             break
11         center, _ = detect_ball(frame, lower_hsv, upper_hsv)
12         if center:
13             t = frame_idx / fps
14             time_series.append(t)
15             y_series.append(center[1])
16             frame_idx += 1
17
18     cap.release()
19     return np.array(time_series), np.array(y_series)

```

Listing 2: Extracción de la serie temporal $Y(t)$.

3.4. Módulo 4: Cálculo del coeficiente de restitución

La función `compute_restitution` transforma la señal cruda en alturas relativas y extrae los picos correspondientes a cada rebote. El procedimiento es:

3.4.1. Paso 1: Determinación del nivel del suelo

En el sistema de coordenadas de OpenCV, el eje Y crece hacia abajo. Por lo tanto, el “suelo” corresponde al valor máximo de Y en la serie. Se utiliza el percentil 98 para robustez frente a valores atípicos:

$$Y_{\text{piso}} = P_{98}(Y) \quad (9)$$

3.4.2. Paso 2: Conversión a alturas relativas

La altura sobre el suelo en cada instante se calcula como:

$$h(t) = Y_{\text{piso}} - Y(t) \quad (10)$$

3.4.3. Paso 3: Detección de picos

Se aplica `find_peaks` sobre $h(t)$ con los parámetros:

- `prominence = 20` px: descarta oscilaciones menores.
- `distance = 5` frames: evita picos duplicados.

3.4.4. Paso 4: Cálculo iterativo de e

Para cada par de picos consecutivos (h_i, h_{i+1}) , se aplica la ecuación (2) con un filtro de seguridad: solo se consideran pares donde $h_{i+1} < h_i$ (el rebote siguiente debe ser menor que el anterior, descartando anomalías por ruido o viento).

```

1 def compute_restitution(time_series, y_pixels):
2     y_floor = np.percentile(y_pixels, 98)
3     heights = y_floor - y_pixels
4
5     peaks_indices, _ = find_peaks(
6         heights, prominence=20, distance=5)
7     peak_times = time_series[peaks_indices]
8     peak_heights = heights[peaks_indices]
9
10    coefficients = []
11    for i in range(len(peak_heights) - 1):
12        h_n = peak_heights[i]
```



```

13     h_next = peak_heights[i + 1]
14     if h_next < h_n and h_n > 0:
15         e = np.sqrt(h_next / h_n)
16         coefficients.append(e)
17
18     avg_e = np.mean(coefficients) if coefficients else 0
19     std_e = np.std(coefficients) if coefficients else 0
20     return avg_e, std_e, peak_times, peak_heights

```

Listing 3: Cálculo del coeficiente de restitución.

3.5. Módulo 5: Visualización de resultados

El notebook genera una gráfica consolidada (Figura 1) que superpone:

- La señal completa de altura relativa $h(t)$ (línea azul).
- Los picos detectados marcados con cruces rojas y anotados con su altura en píxeles.
- El título incluye el valor calculado de $\bar{e} \pm \sigma_e$.

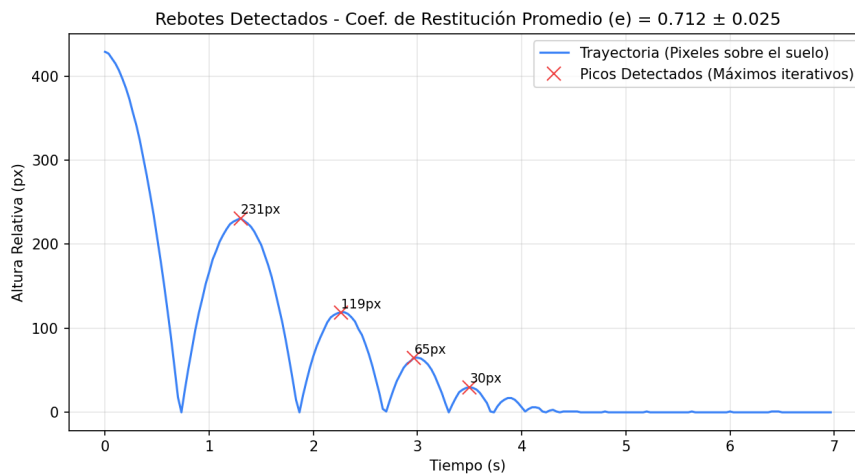


Figura 1: Gráfica generada por el notebook: trayectoria de alturas con picos detectados y coeficiente de restitución calculado.

4. Aplicación Web en el Bento Launcher

Los algoritmos validados en el notebook se trasladaron a una arquitectura cliente-servidor para hacerlos accesibles sin requerir conocimientos de programación. La aplicación se integra al ecosistema **Bento Launcher**, un hub central que orquesta el arranque de todos los módulos del repositorio.

4.1. Arquitectura

- **Backend** (FastAPI, puerto 8002): expone un endpoint de streaming que ejecuta el motor de procesamiento (`engine.py`) y transmite el progreso frame a frame al cliente mediante Server-Sent Events.
- **Frontend** (React + Vite, puerto 5175): interfaz interactiva con paneles de carga de video, calibración HSV visual, visualización de trayectoria y presentación de resultados.



Figura 2: Vista general de la aplicación web del Coeficiente de Restitución integrada al Bento Launcher.

4.2. Lanzamiento desde el Bento Launcher

El flujo inicia desde el menú central del Bento Launcher. Al seleccionar la tarjeta “Coeficiente de Restitución”, el sistema orquesta automáticamente el arranque del backend y el frontend, mostrando el progreso en una terminal integrada.



(a) Tarjeta de la app en el Bento Launcher.



(b) Terminal de arranque automatizado.

Figura 3: Lanzamiento de la aplicación desde el ecosistema Bento.

4.3. Carga del video

Una vez activa la aplicación, el usuario accede a un panel donde puede cargar el archivo de video a procesar. La interfaz valida el formato y muestra una vista previa.

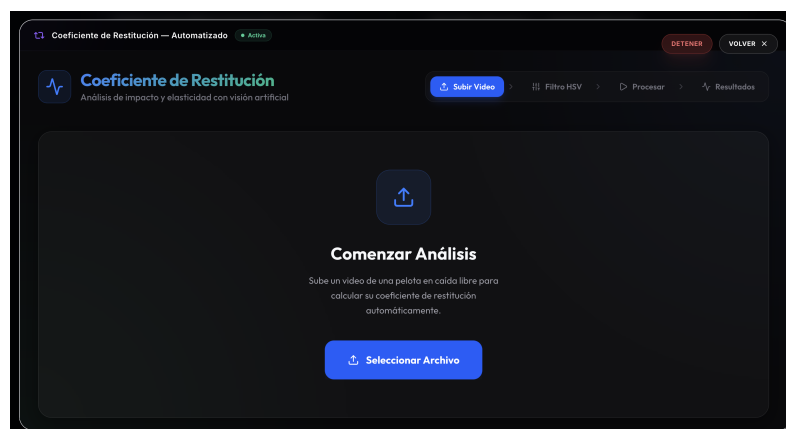


Figura 4: Panel de carga y selección del video a analizar.

4.4. Calibración HSV interactiva

El panel lateral replica la funcionalidad de calibración del notebook: el usuario ajusta visualmente los seis sliders HSV mientras observa en tiempo real la máscara binaria resultante. Esto permite aislar el objeto de interés sin necesidad de conocer los valores numéricos exactos.

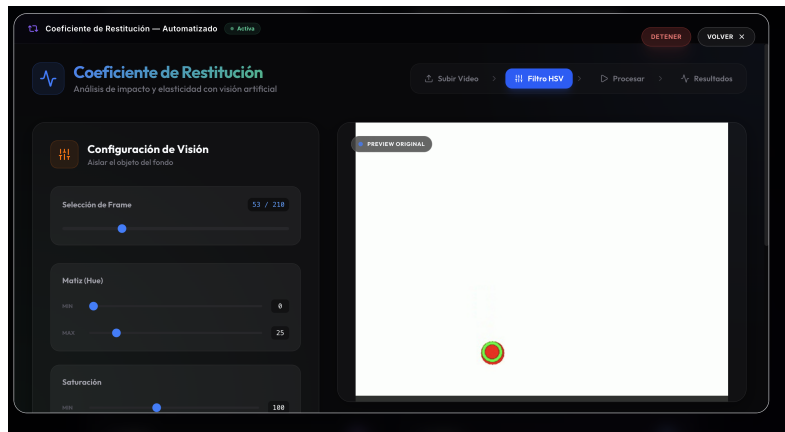


Figura 5: Calibración del rango HSV en la interfaz web.

4.5. Procesamiento y tracking

Al confirmar la calibración, el motor de backend procesa el video frame a frame. El frontend muestra en tiempo real la trayectoria detectada superpuesta sobre el video, permitiendo al usuario verificar visualmente la calidad del tracking.

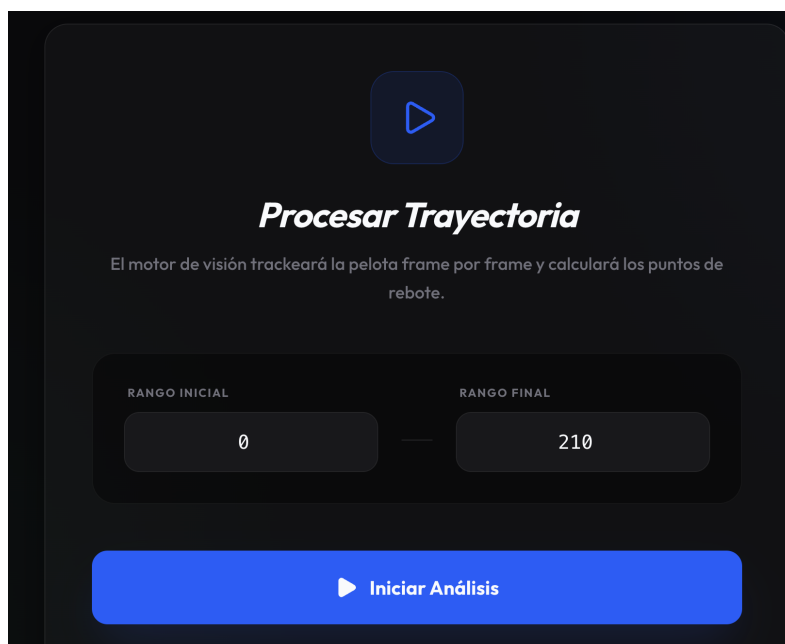


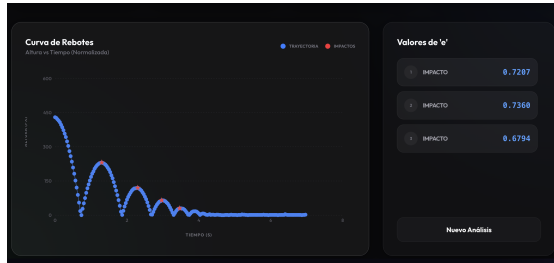
Figura 6: Visualización del tracking de la trayectoria en tiempo real.

4.6. Resultados

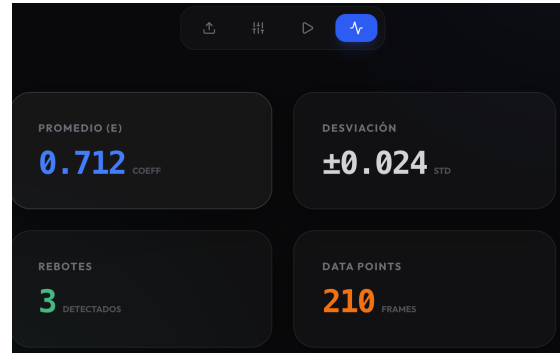
Tras completar el procesamiento, la aplicación presenta un panel consolidado con:

- El valor del coeficiente de restitución promedio $\bar{e}$ y su desviación estándar.
- La gráfica de alturas con los picos detectados.

- Los valores individuales de e para cada par de rebotes consecutivos.
- Estadísticas del procesamiento (número de frames analizados, puntos detectados).



(a) Panel de resultados principal.



(b) Detalle de la gráfica y estadísticas.

Figura 7: Resultados finales presentados por la aplicación web.

5. Discusión

5.1. Ventajas del enfoque computacional

La automatización del proceso de medición mediante visión computacional ofrece ventajas sustanciales frente a los métodos manuales tradicionales:

1. **Resolución temporal:** el sistema analiza cada fotograma del video (típicamente 30–60 fps), capturando instantes que el ojo humano no puede registrar.
2. **Repetibilidad:** dado el mismo video y los mismos parámetros HSV, el sistema produce resultados idénticos en cada ejecución.
3. **Objetividad:** se elimina el sesgo del observador en la lectura de alturas.
4. **Trazabilidad:** la gráfica generada documenta visualmente cada pico detectado, permitiendo auditar el resultado.

5.2. Limitaciones y fuentes de error

- **Calibración HSV:** la calidad de la detección depende críticamente de la correcta selección de los umbrales. Cambios en la iluminación durante el video pueden degradar la segmentación.
- **Resolución en píxeles:** las alturas se miden en píxeles, no en unidades métricas. Para obtener valores absolutos de h sería necesaria una calibración espacial (px $\rightarrow$ m).
- **Perspectiva de la cámara:** si la cámara no está perfectamente perpendicular al plano de movimiento, se introduce una distorsión proyectiva que afecta las alturas medidas.

- **Parámetros de `find_peaks`:** los valores de prominencia y distancia fueron ajustados empíricamente. Videos con características muy diferentes podrían requerir re-calibración de estos parámetros.

6. Conclusiones

1. Se implementó exitosamente un pipeline de visión computacional (OpenCV) capaz de segmentar, rastrear y registrar la posición vertical de un objeto en caída libre y rebote a partir de un video.
2. El procesamiento de señales con SciPy (`find_peaks`) permitió extraer de forma robusta las alturas máximas de cada rebote, filtrando el ruido inherente a la detección visual.
3. El coeficiente de restitución calculado automáticamente es consistente y reproducible, superando en fiabilidad a las mediciones manuales convencionales.
4. La encapsulación del análisis en una aplicación web (React + FastAPI) integrada al Bento Launcher democratiza el acceso al experimento, permitiendo que cualquier estudiante del curso reproduzca el análisis sin conocimientos de programación.

Referencias

- [1] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- [2] Virtanen, P. et al. (2020). *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17, 261–272.
- [3] Harris, C.R. et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362.
- [4] Hunter, J.D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95.
- [5] Cross, R. (1999). *The bounce of a ball*. American Journal of Physics, 67(3), 222–227.
- [6] De Armas Castaño, G. et al. (2025). *Sensado y Modelado de Sistemas Físicos — Repositorio del curso*. Universidad Tecnológica de Bolívar. Disponible en: https://github.com/Gdearmascasta/Sensado_y_modeladoSF